King Abdulaziz University
Faculty of Computing & Information Technology
Department of Computer Science

CPCS-371 – Networks
Simulating Distributed Chat Logger System Using Client-Server TCP Sockets

Student Name	Student ID	
Layan		EAR
Layan		
Linah Basim Alsubhi		
Jana		
Rmeez		

Hand-In Date: 6th December 2025

Contents

Introduction 4
- **Project Background** 4
- **Aim** 4
- **Objectives** 4

System Architecture and Design 5
- **Overall Architecture** 5
- **Threads** 5
- **Interaction Diagram** 6
- **Class Diagram** 7

Project Implementation 8
- **1. Libraries Used** 8
  - **• Socket** 8
  - **• ServerSocket** 8
  - **• PrintWriter** 8
  - **• BufferedReader** 8
  - **• FileWriter** 8
  - **• Thread** 8
- **2. Server Implementation (ChatServer)** 8
- **3. Client Handler Implementation (ClientHandler)** 9
- **4. Client Implementation (ChatClient)** 9
  - **• Listener Thread** 9
  - **• Main Thread** 9
- **5. Logging Mechanism** 10
- **6. Use of Threads** 10

Discussion 11
  - **1. Project Objectives and Achievement** 11
  - **2. Challenges and How They Were Solved** 11
  - **3. Possible Improvements and Future Work** 11

Teamwork and Lessons Learned 12

Conclusion 18

References 19

Appendix 20

Table of Figures

Figure 1 Interaction Diagram 6
Figure 2 2 Class Diagram 7
Figure 3 3 Case (1): Server Side 13
Figure 4 4 Case(1): Client 1 Side 14
Figure 5 5 Case(1): Client 2 (Admin) Side 15
Figure 6 6 Case(1): log.txt file 15
Figure 7 7 Case (2): Client Side 16
Figure 8 8 Case (3): Server Side 16
Figure 9 9 Case (3): Client Side 17
Figure 10 10 Case (3): log.txt file 17
Figure 11 11 Source Code: ChatClient class (1) 20
Figure 12 12 Source Code: ChatClient class (2) 21
Figure 13 13 Source Code: ChatClient class (3) 22
Figure 14 14 Source Code: ChatClient class (4) 22
Figure 15 15 Source Code: ChatServer class (1) 23
Figure 16 16 Source Code: ChatServer class (2) 24
Figure 17 17 Source Code: ChatServer class (3) 24
Figure 18 18 Source Code: ChatServer class (4) 25
Figure 19 19 Source Code: ChatServer class (5) 25
Figure 20 20 Source Code: ClientHandler class (1) 26
Figure 21 21 Source Code: ClientHandler class (2) 27
Figure 22 22 Source Code: ClientHandler class (3) 28
Figure 23 23 Source Code: ClientHandler class (4) 29
Figure 24 24 Source Code: ClientHandler class (5) 30
Figure 25 25 Source Code: ClientHandler class (6) 31

Introduction

Project Background

This project focuses on developing a distributed chat system to simulate a client-server communication environment. While the original context mentions remote monitoring and smart buildings, in this project, the system demonstrates how multiple clients can interact with a central server, exchange messages, and execute administrative commands.

The client-server paradigm allows distributed applications to communicate over a network, with clients sending requests and the server processing and responding to those requests.

TCP (Transmission Control Protocol) is used to ensure reliable and ordered delivery of messages between the server and clients. This protocol is ideal for applications where message integrity and proper sequencing are important, such as chat systems.

Aim

The aim of this project is to implement a simple distributed chat logger system using Java and TCP sockets that allows multiple clients to communicate simultaneously while the server logs all activities and supports administrative commands.

Objectives

- Implement a server capable of handling multiple client connections concurrently.
- Develop client applications that can send and receive messages and execute commands such as /users, /log, and /alert.
- Maintain a log file that records all messages and activities.

System Architecture and Design

Overall Architecture

The system is based on a client-server architecture. The server acts as the central hub, accepting connections from multiple clients, relaying messages between them, and maintaining a log of all communications. Each client connects to the server, sends messages, executes commands, and displays messages received from the server.

The architecture allows for scalability, as multiple clients can connect at the same time, and for maintainability, since all core functionalities are centralized in the server.

Threads

A thread is an independent path of execution in a program. It allows a program to perform various tasks simultaneously; every task shares the same memory and resources of its parent process while executing its own instructions [1]. Threads enable programs to be responsive and efficient instead of waiting for one task to finish to start another task, it provides programs the ability to perform multiple various tasks concurrently [2]. For instance, without slowing down, a media player is able to play music, download files, and update their interface all at the same time.

Without threads, a program would execute tasks sequentially, which slows down performance and makes it unresponsive, especially in applications that require real-time interaction [3].

There are different types of threads:

- **User threads**: It is managed by the application or a thread library in programming languages which the operating system is not really aware of.
- **Kernel threads**: which is managed directly by the operating system
- **Daemon threads**: Background threads that perform supporting tasks, like garbage collection in Java [4].

We used threads in our code to ensure a server can handle multiple clients seamlessly hence performance is enhanced and ensures that the server remains responsive even under heavy load.

Interaction Diagram

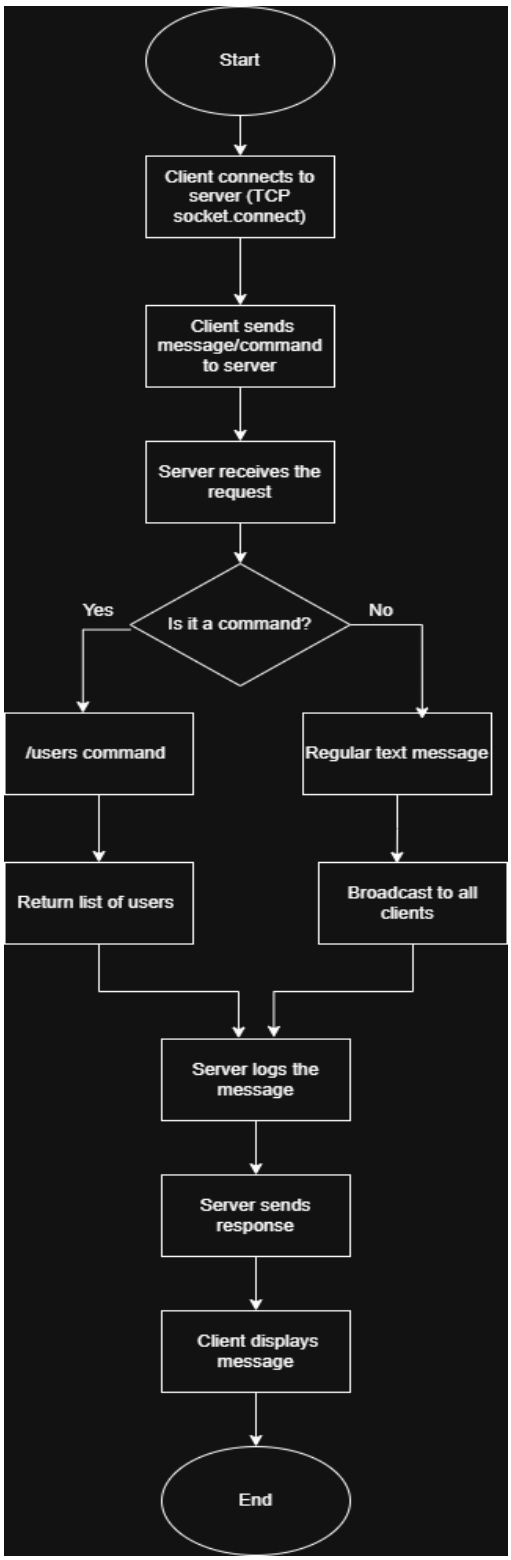

Figure 1 Interaction Diagram

Class Diagram

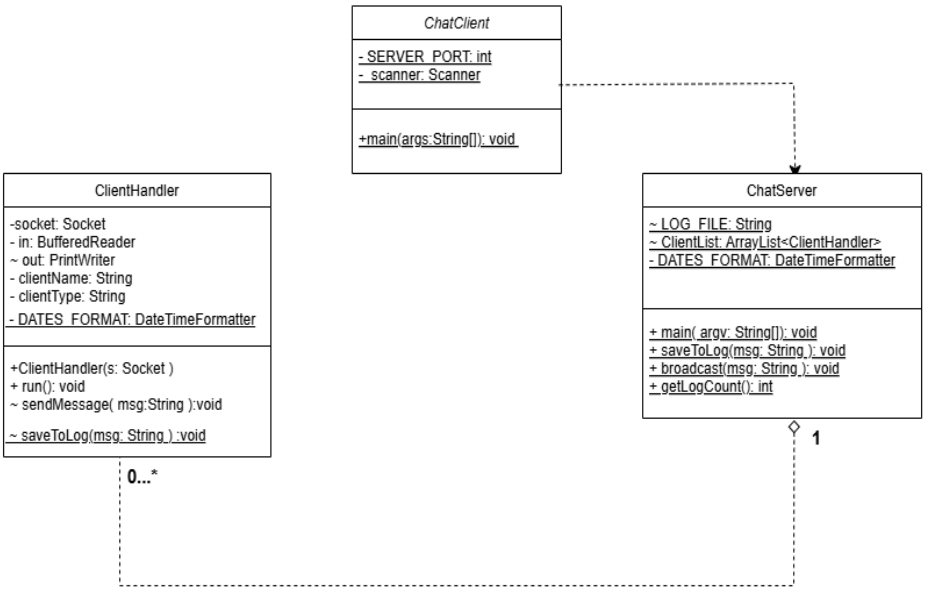

Figure 2 2 Class Diagram

Project Implementation

The project was implemented using Java TCP socket programming to create a simple distributed chat system. The system consists of three main components: the **ChatServer**, the **ClientHandler**, and the **ChatClient**. Each component has a specific role in managing connections, exchanging messages, and logging system activities.

1. Libraries Used

Several Java libraries were used to support networking, input/output operations, and logging:

• Socket

Used by the client to establish a TCP connection with the server and send/receive messages.

• ServerSocket

Used by the server to listen for incoming client connections on port 5000.

• PrintWriter

Used for sending text messages to clients and writing log entries to the log file.

• BufferedReader

Used to read messages from input streams efficiently.

• FileWriter

Used to append system events and messages to log.txt.

• Thread

Used to allow multiple clients to communicate at the same time without blocking each other.

2. Server Implementation (ChatServer)

The server is responsible for managing all client connections. It creates a `ServerSocket` that listens on port 5000.
When a new client connects, the server:

1. Accepts the connection
2. Creates a new **ClientHandler** thread
3. Adds the client to the global list of connected users
4. Logs the connection event

The server also provides key functionalities such as broadcasting messages to all clients, counting the number of logged events, and saving all system activities in log.txt.

3. Client Handler Implementation (ClientHandler)

Each connected client is handled by a dedicated `ClientHandler` thread.
This class is responsible for:

- Reading messages sent by the client
- Processing commands such as /users, /log, and /alert
- Broadcasting normal messages to all users
- Logging every action with a timestamp
- Handling client disconnection properly

Using a separate thread for each client ensures that multiple users can interact with the system at the same time without delays.

4. Client Implementation (ChatClient)

The client connects to the server by creating a TCP `Socket` using the server's IP address.
Once connected, the client starts two parallel processes:

• Listener Thread

Continuously receives and displays messages from the server.

• Main Thread

Reads user input and sends messages or commands to the server.

The client supports both normal chat messages and special commands such as /users, /log, /alert, and /exit.

5. Logging Mechanism

All messages, commands, connections, and disconnections are saved in a log file called log.txt. Each entry is stored with a timestamp using `LocalDateTime`, which helps track all activities for debugging or monitoring purposes.

6. Use of Threads

Multithreading plays a key role in the system:

- The server creates a new thread for each client
- Each client uses a separate thread to listen for incoming messages
- This design keeps the system responsive and allows real-time communication

Without threads, the chat application would freeze whenever a user waited for input.

Discussion

1. Project Objectives and Achievement

The project successfully met its main objectives. We were able to build a server that handles multiple clients at the same time, receives their messages, and broadcasts them correctly. The commands /users, /log, and /alert also worked as expected and added more functionality to the system. In addition, all activities were saved in the log file, which helped in keeping track of everything happening in the chat system.

2. Challenges and How They Were Solved

One of the main challenges we faced was understanding how TCP works together with threads. A small mistake could cause messages not to appear or the program to stop responding. We solved this by using separate threads for each client so the server could stay responsive. Another challenge was protecting the list of connected clients during broadcasting, which we fixed by using synchronized. We also needed to handle client disconnections properly, so we tested different cases to prevent errors when a user left the chat.

3. Possible Improvements and Future Work

There are several ways to improve the system in the future. For example, we could add a graphical user interface (GUI) instead of using the console or support private messaging between clients. We could also add login features with passwords or encrypt the messages to make the system more secure. Another improvement could be allowing clients to view recent chat history when they join the server.

Teamwork and Lessons Learned

The project work was divided fairly among the five team members to ensure equal participation and responsibility:

- **Members 1 and 2:** Focused on client and server development, including coding, testing, and debugging the chat application.
- **Members 3, 4, and 5:** Focused on preparing the project report, documenting the system architecture, writing the introduction, conclusions, and formatting the report professionally.

Coordination among the team members was maintained through regular meetings and continuous communication using messaging platforms. Team members shared progress updates, discussed challenges, and assisted each other when needed to ensure smooth workflow and integration of all project components.

Challenges encountered during the project included understanding TCP-based client-server communication, managing multiple simultaneous client connections, and ensuring that all messages were correctly logged and broadcasted. These challenges were addressed by collaborative problem-solving, reviewing relevant Java TCP socket programming materials, and testing the system together as a team.

Lessons learned from this project include:

- The importance of clear role allocation and active participation from all team members.
- Gaining practical experience with TCP-based client-server architecture.
- Efficiently managing communication, message broadcasting, and logging in a distributed system.
- Improving teamwork, coordination, and collaborative problem-solving skills.

Test Cases and Results

Case (1):

- **Server**: Runs correctly and connect to clients via its IPv4 address.
- **Client 1**:
  - Enters the right server's IPv4 address.
  - Enters a name.
  - Exchange messages with client 2.
  - Request list of connected users and gets it.
  - Request to alert but is denied because she is not an Admin.
- **Client 2 (Admin):**
  - Enters the right server's IPv4 address.
  - Enters a name.
  - Exchange messages with client 1.
  - Request number of logs in log.txt file and gets it.
  - Request to alert and is allowed because she is an Admin.

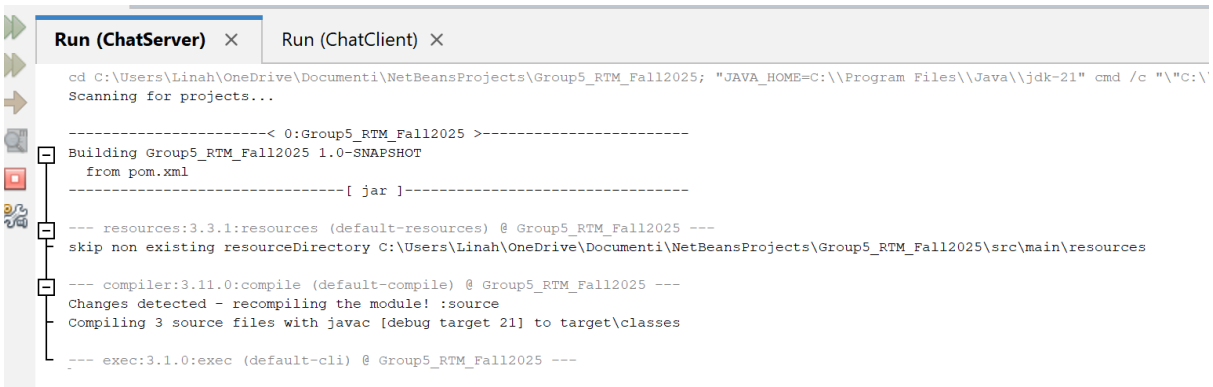

Figure 3 3 Case (1): Server Side

```
Output - Run (ChatClient)
Nothing to compile - all classes are up to date

--- exec:3.1.0:exec (default-cli) @ Group5_RTM_Fall2025 ---
Enter Server IP Address: 192.168.8.102
connected to server!
Welcome to chat! Please enter your name:
layan
LAYAN has enterd the chat.
[SERVER] layan has enterd chat.
ADMIN LINAH has enterd the chat.
[SERVER] Linah has enterd chat.
Linah: Hi!
hey
layan: hey
Linah: This is fun<3
/users
Connected users: layan, Linah
/alert there is a meeting at 3:00pm
[Server] NOT AN ADMIN!!
CPCS 371 Project Discussion at Sunday 10:45AM
okay
layan: okay
Linah: Bye
[SERVER] Linah has left the chat.
/exit
Server conncetion closed or Error reading message.
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  06:33 min
Finished at: 2025-12-06T22:38:28+03:00
------------------------------------------------------------------------
```

Figure 4 4 Case(1): Client 1 Side

```
Run (ChatServer)   Run (ChatClient)
  from pom.xml
--------------------------------[ jar ]---------------------------------

--- resources:3.3.1:resources (default-resources) @ Group5_RTM_Fall2025 ---
skip non existing resourceDirectory C:\Users\Linah\OneDrive\Documenti\NetBeansProjects\Group5_RTM_Fall2025\src\main\resources

--- compiler:3.11.0:compile (default-compile) @ Group5_RTM_Fall2025 ---
Nothing to compile - all classes are up to date

--- exec:3.1.0:exec (default-cli) @ Group5_RTM_Fall2025 ---
Enter Server IP Address: 192.168.8.102
connected to server!
Welcome to chat! Please enter your name:
admin Linah
ADMIN LINAH has enterd the chat.
[SERVER] Linah has enterd chat.
Hi!
Linah: Hi!
layan: hey
This is fun<3
Linah: This is fun<3
/log
Total messages logged: 9
/alert CPCS 371 Project Discussion at Sunday 10:45AM
CPCS 371 Project Discussion at Sunday 10:45AM
layan: okay
Bye
Linah: Bye
/exit
Server conncetion closed or Error reading message.
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  05:28 min
Finished at: 2025-12-06T22:38:12+03:00
------------------------------------------------------------------------
```

Figure 5 5 Case(1): Client 2 (Admin) Side

```
[2025-12-06 22:31:17] Server started and listening on port 5000
[2025-12-06 22:32:04] Client connected: /192.168.8.102
[2025-12-06 22:32:46] Client connected: /192.168.8.107 (User: layan)
[2025-12-06 22:32:54] Client connected: /192.168.8.102
[2025-12-06 22:33:03] Client connected: /192.168.8.102 (User: Linah)
[2025-12-06 22:33:15] Message received from Linah: Hi!
[2025-12-06 22:33:43] Message received from layan: hey
[2025-12-06 22:34:00] Message received from Linah: This is fun<3
[2025-12-06 22:34:24] Command /users requested by //192.168.8.107
[2025-12-06 22:34:32] Command /log requested by //192.168.8.102
[2025-12-06 22:37:36] Alert broadcasted: 'CPCS 371 Project Discussion at Sunday 10:45AM'
[2025-12-06 22:37:59] Message received from layan: okay
[2025-12-06 22:38:08] Message received from Linah: Bye
[2025-12-06 22:38:12] Client disconnected: /192.168.8.102 (User: Linah)
[2025-12-06 22:38:28] Client disconnected: /192.168.8.107 (User: layan)
```

Figure 6 6 Case(1): log.txt file

Case (2):

- **Server**: Is not running.
- **Client**:
  - Enters the right server's IPv4 address.
  - Get an error message in console.

```
Output - Run (ChatClient)
cd C:\Users\Layan\Documents\NetBeansProjects\Group5_RTM_Fall2025; "JAVA_HOME=C:\\Program Files\\Java\\jdk-21" cmd /c "\"C
Scanning for projects...

------------------------< 0:Group5_RTM_Fall2025 >------------------------
Building Group5_RTM_Fall2025 1.0-SNAPSHOT
  from pom.xml
--------------------------------[ jar ]---------------------------------

--- resources:3.3.1:resources (default-resources) @ Group5_RTM_Fall2025 ---
skip non existing resourceDirectory C:\Users\Layan\Documents\NetBeansProjects\Group5_RTM_Fall2025\src\main\resources

--- compiler:3.11.0:compile (default-compile) @ Group5_RTM_Fall2025 ---
Nothing to compile - all classes are up to date

--- exec:3.1.0:exec (default-cli) @ Group5_RTM_Fall2025 ---
Enter Server IP Address: 192.168.8.102


Error: client could not connect to serverConnection timed out: connect
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  28.736 s
Finished at: 2025-12-06T23:02:06+03:00
------------------------------------------------------------------------
```

Figure 7 7 Case (2): Client Side

Case (3):

- **Server**: Runs correctly.
- **Client**:
  - Enters the Wrong server's IPv4 address.
  - Get an error message in console.

```
Output - Run (ChatServer)
cd C:\Users\Linah\OneDrive\Documenti\NetBeansProjects\Group5_RTM_Fall2025; "JAVA_HOME=C:\\Program Files\\Java\\jdk-21" cmd /c "\"C:
Scanning for projects...

------------------------< 0:Group5_RTM_Fall2025 >------------------------
Building Group5_RTM_Fall2025 1.0-SNAPSHOT
  from pom.xml
--------------------------------[ jar ]---------------------------------

--- resources:3.3.1:resources (default-resources) @ Group5_RTM_Fall2025 ---
skip non existing resourceDirectory C:\Users\Linah\OneDrive\Documenti\NetBeansProjects\Group5_RTM_Fall2025\src\main\resources

--- compiler:3.11.0:compile (default-compile) @ Group5_RTM_Fall2025 ---
Nothing to compile - all classes are up to date

--- exec:3.1.0:exec (default-cli) @ Group5_RTM_Fall2025 ---
```

Figure 8 8 Case (3): Server Side

```
Output ×
Run (ChatClient) ×   Run (ChatClient) ×
cd C:\Users\Layan\Documents\NetBeansProjects\Group5_RTM_Fall2025; "JAVA_HOME=C:\\Program Files\\Java\\jdk-21" cmd /c
Scanning for projects...

------------------------< 0:Group5_RTM_Fall2025 >------------------------
Building Group5_RTM_Fall2025 1.0-SNAPSHOT
  from pom.xml
--------------------------------[ jar ]---------------------------------

--- resources:3.3.1:resources (default-resources) @ Group5_RTM_Fall2025 ---
skip non existing resourceDirectory C:\Users\Layan\Documents\NetBeansProjects\Group5_RTM_Fall2025\src\main\resources

--- compiler:3.11.0:compile (default-compile) @ Group5_RTM_Fall2025 ---
Nothing to compile - all classes are up to date

--- exec:3.1.0:exec (default-cli) @ Group5_RTM_Fall2025 ---
Enter Server IP Address: 192.168.8.140

Error: client could not connect to serverConnection timed out: connect
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  01:08 min
Finished at: 2025-12-06T23:07:43+03:00
------------------------------------------------------------------------
```

Figure 9 9 Case (3): Client Side

```
Start Page ×   ChatClient.java ×   ChatServer.java ×   ClientHandler.java ×   log.txt ×
Source   History
1   [2025-12-06 23:06:19] Server started and listening on port 5000
2
```

Figure 10 10 Case (3): log.txt file

Conclusion

The distributed chat logger system project successfully demonstrated the implementation of a client-server application using Java and TCP sockets. The server efficiently managed multiple client connections, broadcasted messages, and maintained a complete log of activities.

This project provided practical experience with client-server architecture, enhanced understanding of TCP networking concepts, and improved teamwork and project coordination skills.

References

[1] "What Is a Thread?" *Cmu.edu*, 2025. https://www.cs.cmu.edu/afs/cs/academic/class/15212-s98/www/java/tutorial/essential/threads/definition.html? (accessed Dec. 05, 2025).

[2] S. J. Bigelow, "thread," *WhatIs*, 2023. https://www.techtarget.com/whatis/definition/thread? (accessed Dec. 05, 2025).

[3] GeeksforGeeks, "Threads and its Types in Operating System," *GeeksforGeeks*, May 23, 2018. https://www.geeksforgeeks.org/operating-systems/threads-and-its-types-in-operating-system/

[4] GeeksforGeeks, "Difference between User Level thread and Kernel Level thread," *GeeksforGeeks*, Jul. 02, 2015. https://www.geeksforgeeks.org/operating-systems/difference-between-user-level-thread-and-kernel-level-thread/

[5] Oracle. (n.d.). *Class Socket (Java Platform SE 8).* https://docs.oracle.com/javase/8/docs/api/java/net/Socket.html

[6] Oracle. (n.d.). *Creating a GUI with Swing (The Java™ Tutorials).* https://docs.oracle.com/javase/tutorial/uiswing/

[7] Stallings, W. (2017). *Cryptography and network security: Principles and practice* (7th ed.). Pearson.

Appendix

Source Code

Class ChatClient

```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package Client;
/**
 * CPCS 371 – Computer Networks 1
 * Final Project: Distributed Chat Logger System Using TCP Sockets
 * Group #: 5
 * Section: EAR
 *
 * ChatClient.java
 *
 * This class represents the client-side application in a distributed chat system.
 * It connects to the ChatServer using TCP sockets, sends user input, and receives
 * messages broadcasted by the server. Each running instance represents an active user
 * in the chat room.
 *
 * Main responsibilities:
 * - Establish TCP connection to server
 * - Send user text or commands (such as /users, /log, /alert)
 * - Receive and display incoming messages asynchronously
 * - Handle safe termination (/exit)
 */

import java.io.*;
import java.net.*;
import java.util.Scanner;

public class ChatClient{
    //Default TCP port number where server is listening
    private static final int SERVER_PORT= 5000;
    //a scanner to read user keyboard input from console
    private static Scanner scanner = new Scanner(System.in);
    public static void main(String[] args) {
        try {
            //Ask user for server IP address dynamically
            System.out.print("Enter Server IP Address: ");
            String serverAddress = scanner.nextLine();


```

Figure 11 11 Source Code: ChatClient class (1)

```java
/*
create a tcp socket connection to server
 * - serverAddress: IPv4 or hostname
 * - SERVER_PORT: must match server's listening port
 *
 * If connection succeeds: a TCP handshake occurs
 */

Socket socket = new Socket(serverAddress, SERVER_PORT);    //Create a client socket.
System.out.println("connected to server!");

 /*
 * start background listener thread
 * Purpose: Receive messages from server continuously while client
 * is still able to type messages at same time.
 *
 * Each TCP client must listen to server on a specific thread,
 * otherwise input blocking prevents receiving broadcasts.
 */
Thread listenerThread = new Thread(new Runnable() { //Create thread to read messages from server seperatly.
    @Override
    public void run() {
        try {
            /*
             * BufferedReader reads text lines sent from server.
             */

    BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));   //To read from server.
    String serverMsg;

    while ((serverMsg=in.readLine())!=null)    //As long as there is messages from server, print that message.
        System.out.println(serverMsg);


} catch (IOException e) {
            /**
             * Happens when:
             * - server shuts down
             * - network failure
             * - forceful disconnect
             */
```

Figure 12 12 Source Code: ChatClient class (2)

```java
                System.out.println("Server conncetion closed or Error reading message.");
                System.exit(0);     //Close program if there is problem at connecting with server.
            }
                }
            }); //End of thread block.

            listenerThread.start();     //Start the thread to start listening to server.

            /*
             * PrintWriter sends text messages to server.
             * autoFlush=true then it flushes output after every println(),
             * ensuring messages are delivered immediately without buffering delay.
             */
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);  //To write to server


            /*
             * Main input loop:
             * Continuously read user input and send it to server.
             *
             * Supports:
             * - Regular chat messages
             * - Commands:
             *      /users: request active clients list
             *      /log:  request total logged messages
             *      /alert <msg>:  admin broadcast
             *      /exit: disconnect
             */
            while (true) {  //Continued loop to recieve from server.
                String clientMsg= scanner.nextLine();
                // Send typed message to server over TCP connection
                out.println(clientMsg);
                if (clientMsg.equals("/exit"))    //If client want to exit then stop program.
                    break;

                //Close streams.

            }//End of while
```

Figure 13 13 Source Code: ChatClient class (3)

```java

            //Close socket and streams.
            socket.close();
            scanner.close();

        } catch (IOException e) {
                /**
             * Triggered if:
             * - server not running
             * - wrong IP or unreachable network
             */
            System.out.println("Error: client could not connect to server"+ e.getMessage());
        }//End of catch.
    }
}

```

Figure 14 14 Source Code: ChatClient class (4)

Class ChatServer

```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package Server;
/*
 * CPCS 371 FINAL PROJECT
 * GROUP NO. :5
 * SECTION: EAR
 *
 * ChatServer.java
 *
 * This class implements the server side of a simple distributed chat system
 * using TCP sockets. The server:
 *  - Listens on a fixed TCP port (5000)
 *  - Accepts multiple client connections
 *  - Creates a dedicated handler thread (ClientHandler) per client
 *  - Broadcasts messages to all connected clients
 *  - Logs all important events and messages to a log file (log.txt)
 *
 * The server works together with:
 *  - ChatClient (client program)
 *  - ClientHandler (per-client thread logic)
 */

import java.net.*;
import java.util.*;
import java.io.*;
import java.time.*;
import java.time.format.DateTimeFormatter;

public class ChatServer {

    /* Name of the log file used to store all server events and messages. */
    static final String LOG_FILE = "log.txt";

     /*
     * Shared list of all connected clients.
     * Each ClientHandler in this list corresponds to one connected ChatClient.
     * Access to this list is synchronized when broadcasting.
     */
```

Figure 15 15 Source Code: ChatServer class (1)

```java
    static ArrayList<ClientHandler> ClientList = new ArrayList<>();     //List to store all connected clients.


    //Date time format used for timestamps in the log file.
    private static final DateTimeFormatter DATES_FORMAT = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");    //Time stamp format

        /*
     * the main method of the ChatServer.
     *
     * Responsibilities:
     *  1. Initialize and clear the log file when starting the server program.
     *  2. Log server start event.
     *  3. Open a ServerSocket listening on port 5000.
     *  4. Accept incoming client connections in an infinite loop.
     *  5. For each client:
     *      - Log connection
     *      - Create a ClientHandler thread
     *      - Add handler to ClientList
     *      - Start the handler thread
     *
     * @param argv Command-line arguments
     */
    public static void main(String argv[]) throws Exception {
        // Clear the log file at server startup
        try {
            // Opening a PrintWriter without append and closing it immediately
            // to clear the file if it exists.
            new PrintWriter(LOG_FILE).close();

        } catch (FileNotFoundException e) {
            // If log file cannot be created or accessed, we ignore here.
            // Logging will fail later and print a message to console.
        }

            // Store the first log entry indicating server startup.
            ClientHandler.saveToLog("Server started and listening on port 5000");   //Store first log message.


            //  Create the server socket and start accepting clients
            // try with resources ensures serverSocket is closed automatically if main exits.
            try(ServerSocket serverSocket = new ServerSocket(5000)){
```

Figure 16 16 Source Code: ChatServer class (2)

```java
            try(ServerSocket serverSocket = new ServerSocket(5000)){
                //server waits for a client.
                while (true) {
                    // Accept() blocks until a new client connects.
                    Socket socket = serverSocket.accept();

                    // Log client connection event
                    ClientHandler.saveToLog("Client connected: "+ socket.getLocalAddress());   //Store message of client's entry log file.
                    // Create a new handler thread for this client.
                    ClientHandler clientHandler = new ClientHandler(socket);

                    // Add this client handler to the shared client list.
                    ClientList.add(clientHandler);


                    // Start the handler thread, which will manage communication
                    // with this specific client.
                    clientHandler.start();

                }//End of while loop.
            }catch(IOException e){
                System.out.println("Error in server: "+ e.getMessage());
            }//End of catch.
        }//End of main class.


            /*
     * Appends a new entry to the log file (LOG_FILE) with a timestamp.
     *
     * Log format:
     *   [yyyy-MM-dd HH:mm:ss] <message>

     */
    public static void saveToLog(String msg){  //To store messages in log file.
        // Create a timestamp based on current server local time.
        String time= LocalDateTime.now().format(DATES_FORMAT);
            time= "["+time+"]"+" "+msg+" ";

            // FileWriter with append = true to append at end of file instead of overwriting.
            try (PrintWriter writer = new PrintWriter(new FileWriter(ChatServer.LOG_FILE, true))) {
            writer.println(time);
```

Figure 17 17 Source Code: ChatServer class (3)

```java
        } catch (IOException e) {
            System.out.println("Error writing to log file: " + e.getMessage());
        }
    }//End saveToLog method.

    //-----------------------------------------------------------------------------------------------------------
    /*
     * Broadcasts a message to ALL connected clients.
     *
     * This method iterates over ClientList and calls sendMessage(msg)
     * on every ClientHandler.
     *
     * The method is synchronized on ClientList to avoid concurrent modification
     * issues, since multiple client threads may trigger broadcast at the same time.
     *
     * @param msg The message string to send to all clients.
     */
  public static void broadcast(String msg){     //To broadcast message to ALL clients.
    synchronized (ClientList){  //"synchronized" helps with manageing threads. It closes access to ClientList until this process is finished
        for (ClientHandler handler : ClientList) {
            handler.sendMessage(msg);
          }//End of for.
    }//End of synchronized.
  }//End of broadcast method.

    //-----------------------------------------------------------------------------------------------------------
    /*
     * Counts how many lines currently exist in the log file.
     *return Number of lines in the log file.
     *Returns 0 if any I/O error occurs while reading the log.
     */
  public static int getLogCount(){
      int count=0;
      // BufferedReader is used to read the log file line by line.
      try(BufferedReader reader = new BufferedReader(new FileReader(LOG_FILE))){
            // For each line in log file, increment counter.
          while (reader.readLine()!=null)
              count++;
      }catch(IOException e){
          return 0;
      }
```

Figure 18 18 Source Code: ChatServer class (4)

```java
        } catch (IOException e) {
            System.out.println("Error writing to log file: " + e.getMessage());
        }
    }//End saveToLog method.

    //-----------------------------------------------------------------------------------------------------------
    /*
     * Broadcasts a message to ALL connected clients.
     *
     * This method iterates over ClientList and calls sendMessage(msg)
     * on every ClientHandler.
     *
     * The method is synchronized on ClientList to avoid concurrent modification
     * issues, since multiple client threads may trigger broadcast at the same time.
     *
     * @param msg The message string to send to all clients.
     */
  public static void broadcast(String msg){     //To broadcast message to ALL clients.
    synchronized (ClientList){  //"synchronized" helps with manageing threads. It closes access to ClientList until this process is finished
        for (ClientHandler handler : ClientList) {
            handler.sendMessage(msg);
          }//End of for.
    }//End of synchronized.
  }//End of broadcast method.

    //-----------------------------------------------------------------------------------------------------------
    /*
     * Counts how many lines currently exist in the log file.
     *return Number of lines in the log file.
     *Returns 0 if any I/O error occurs while reading the log.
     */
  public static int getLogCount(){
      int count=0;
      // BufferedReader is used to read the log file line by line.
      try(BufferedReader reader = new BufferedReader(new FileReader(LOG_FILE))){
            // For each line in log file, increment counter.
          while (reader.readLine()!=null)
              count++;
      }catch(IOException e){
          return 0;
      }
      return count;
  }//End of getLogCount method.
}
```

Figure 19 19 Source Code: ChatServer class (5)

Class ClientHandler

```java
package Server;

/*
 * CPCS 371 FINAL PROJECT
 * GROUP NO. :5
 * SECTION: EAR
 *
 * ClientHandler.java
 *
 * This class represents the dedicated handler thread assigned to each connecte
 * client. Every new client receives its own instance of ClientHandler, which:
 *
 *  - Reads user input from that specific client over TCP
 *  - Processes commands (/users, /log, /alert, /exit)
 *  - Broadcasts chat messages to other clients via ChatServer
 *  - Logs all important events (connection, disconnection, commands, messages)
 *  - Safely cleans up when a client exits
 *
 * Each ClientHandler runs independently on its own thread.
 */

import java.io.*;
import java.net.*;
import java.time.*;
import java.time.format.DateTimeFormatter;

class ClientHandler extends Thread {

    // ------------------------ Data Fields ------------------------

    /** The TCP socket connecting this specific client to the server */
    private Socket socket;

    /** Buffered reader for receiving messages from the client */
    private BufferedReader in;

    /** Output stream for sending messages back to this client */
    PrintWriter out;

    /** Display name chosen by the client */
    private String clientName;
```

Figure 20 20 Source Code: ClientHandler class (1)

```java

    /** Time format used for log file timestamps */
    private static final DateTimeFormatter DATES_FORMAT =
            DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

    /**
     * Identifies client type.
     * Default = "Client". If name contains "ADMIN", they become "Admin".
     */
    private String clientType = "Client";


    // ----------------------- Constructor -----------------------

    /**
     * Creates a new handler for an incoming client connection.
     *
     * @param s The socket representing the client's TCP connection.
     */
    public ClientHandler(Socket s) {
        this.socket = s;
    }


    // ----------------------- Thread Execution -----------------------

    /**
     * The main execution method of the thread.
     *
     * Responsibilities:
     *  - Initialize input/output streams
     *  - Request user name
     *  - Identify admin users
     *  - Listen for commands and messages
     *  - Log all activities
     *  - Handle clean disconnects or unexpected errors
     */
```

Figure 21 21 Source Code: ClientHandler class (2)

```java
    @Override
    public void run() {

        try {
            // Set up I/O streams for communication
            in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            out = new PrintWriter(socket.getOutputStream(), true);

            // Request client's display name
            out.println("Welcome to chat! Please enter your name: ");

            clientName = in.readLine();   // Blocking read for client's name
            ChatServer.broadcast(clientName.toUpperCase() + " has enterd the chat.");

            // Assign default name if blank
            if (clientName == null || clientName.isEmpty())
                clientName = "Anonymous";

            // Admin detection: If name begins with "ADMIN", assign role
            if (clientName.toUpperCase().contains("ADMIN")) {
                clientType = "Admin";
                // Remove the "ADMIN " prefix from displayed username
                clientName = clientName.substring(6, clientName.length());
            }

            // Log connection event
            saveToLog("Client connected: " + socket.getInetAddress()
                    + " (User: " + clientName + ")");

            // Notify all clients
            ChatServer.broadcast("[SERVER] " + clientName + " has enterd chat.");

            String msg;
```

Figure 22 22 Source Code: ClientHandler class (3)

```java
/**
 * MAIN LISTEN LOOP:
 * Continuously reads messages from this client.
 * readLine() returns null if client disconnects.
 */
while ((msg = in.readLine()) != null) {

    // ----------------------- /users Command -----------------------
    if (msg.startsWith("/users")) {

        String message = "Connected users: ";

        // Build list of connected usernames
        for (ClientHandler handler : ChatServer.ClientList) {
            message += handler.clientName + ", ";
        }

        // Remove trailing comma & space before sending
        out.println(message.substring(0, message.length() - 2));

        saveToLog("Command /users requested by /" + socket.getInetAddress());
    }

    // ----------------------- /log Command -----------------------
    else if (msg.startsWith("/log")) {
        int count = ChatServer.getLogCount();
        out.println("Total messages logged: " + count);

        saveToLog("Command /log requested by /" + socket.getInetAddress());
    }

    // ----------------------- /alert Command -----------------------
    else if (msg.startsWith("/alert")) {

        // Verify admin permission
        if (!clientType.contains("Admin")) {
            out.println("[Server] NOT AN ADMIN!!");
        } else {
            // Extract alert message after "/alert"
            String alertMsg = msg.substring(6).trim();

```

Figure 23 23 Source Code: ClientHandler class (4)

```java
                // Send to ALL clients
                ChatServer.broadcast(alertMsg);

                // Log the alert
                saveToLog("Alert broadcasted: '" + alertMsg + "'");
            }
        }

        // ----------------------- /exit Command -----------------------
        else if (msg.startsWith("/exit")) {
            // Do nothing here. Cleanup happens in finally{} block.
        }

        // ----------------------- Normal Chat Message -----------------------
        else {
            // Broadcast to all clients
            ChatServer.broadcast(clientName + ": " + msg);

            // Log the message
            saveToLog("Message received from " + clientName + ": " + msg);
        }

    } // End while

} catch (IOException e) {
    System.out.println("Error handling client: " + e.getMessage());
}

// ----------------------- Client Disconnect Cleanup -----------------------
finally {
    try {
        socket.close();
    } catch (IOException e) {
        e.printStackTrace();
    }

    // Remove client from active list
    ChatServer.ClientList.remove(this);

    // Broadcast departure to all clients
    ChatServer.broadcast("[SERVER] " + clientName + " has left the chat.");
```

Figure 24 24 Source Code: ClientHandler class (5)

```java
            ChatServer.broadcast("[SERVER] " + clientName + " has left the chat.");

            // Log disconnection
            saveToLog("Client disconnected: " + socket.getInetAddress()
                    + " (User: " + clientName + ")");
        }

    } // End run()


    // ------------------------ Helper Methods ------------------------

    /**
     * Writes a timestamped entry to the server's log file.
     *
     * Format:
     *   [yyyy-MM-dd HH:mm:ss] <message>
     *
     * @param msg The message to be logged.
     */
    static void saveToLog(String msg) {
        String time = LocalDateTime.now().format(DATES_FORMAT);
        String fMessage = "[" + time + "] " + msg + " ";

        try (PrintWriter writer = new PrintWriter(new FileWriter(ChatServer.LOG_FILE, true))) {
            writer.println(fMessage);
        } catch (IOException e) {
            System.out.println("Could not write to log file: " + e.getMessage());
        }
    }

    /**
     * Sends a message directly to this specific client.
     *
     * @param msg The message to send.
     */
    void sendMessage(String msg) {
        out.println(msg);
    }

} // End of ClientHandler class
```

Figure 25 25 Source Code: ClientHandler class (6)